



Análisis de diseño y mejoras visuales

Animaciones · Microinteracciones · Scroll effects · Conversión · Retención

Junio 2026 · Confidencial

Base sólida	0 animaciones scroll	0 transiciones página
Microinteracciones mínima	Dark mode OK	Safe area iOS OK

KAVIRO — ANÁLISIS DE DISEÑO Y MEJORAS VISUALES

Fecha: Junio 2026

Base: Design tokens sólidos (Coral #F87171, Inter, dark mode bien implementado)

Estado actual del diseño — qué hay y qué falta

Lo que ya está bien

Sistema de colores coherente — `--brand: #F87171`` aplicado de forma consistente en toda la app

Dark mode bien implementado — variables CSS distintas para light/dark, transición suave `0.2s ease``

Tipografía correcta — Inter + SF Pro Display, font-smoothing activado

Bordes redondeados generosos — `rounded-2xl`` / `rounded-3xl`` dan sensación moderna

Safe area iOS — correctamente implementada con variables CSS

Gradientes en el hero del resumen — la tarjeta oscura con blur circles da nivel premium

Blur decorativo en la landing — el radial gradient coral en el hero da calidez sin ser agresivo

Lo que falta — diagnóstico honesto

0 animaciones de scroll — ningún elemento entra con fade, slide o scale al llegar al viewport

0 transiciones de página — solo existe la barra de progreso global (GlobalRouteLoading)

Tarjetas de viaje estáticas — sin microinteracciones al hacer hover salvo un shadow mínimo

Landing sin ritmo visual — todas las secciones tienen el mismo peso visual, no hay jerarquía de atención

Pricing sin efecto "elegir" — el plan Premium tiene el badge "Popular" pero sin animación de aparición

Hero de la landing sin movimiento — el mockup de la app en el lado derecho es estático

Formularios sin feedback visual rico — el foco tiene ring coral pero el éxito/error es básico

Mejoras recomendadas — de mayor a menor impacto

1. Animaciones de scroll — "scroll-reveal" en la landing

Impacto en conversión: muy alto. Los usuarios que ven elementos aparecer mientras hacen scroll tienen un 40-60% más de tiempo en página.

Qué añadir en `globals.css`:

```
/* Elementos que entran al llegar al viewport */
@keyframes slide-up {
  from { opacity: 0; transform: translateY(24px); }
  to   { opacity: 1; transform: translateY(0); }
}

@keyframes fade-in {
  from { opacity: 0; }
  to   { opacity: 1; }
}

@keyframes scale-in {
  from { opacity: 0; transform: scale(0.95); }
  to   { opacity: 1; transform: scale(1); }
}

.reveal { opacity: 0; }
.reveal.visible { animation: slide-up 0.5s ease-out forwards; }
.reveal-fade.visible { animation: fade-in 0.4s ease-out forwards; }
.reveal-scale.visible { animation: scale-in 0.4s ease-out forwards; }

/* Delays para escalonar elementos en grid */
.delay-1 { animation-delay: 0.1s; }
.delay-2 { animation-delay: 0.2s; }
.delay-3 { animation-delay: 0.3s; }
.delay-4 { animation-delay: 0.4s; }
```

Hook `useScrollReveal` (20 líneas):

```
// hooks/useScrollReveal.ts
import { useEffect } from "react";

export function useScrollReveal() {
  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => entries.forEach((e) => {
        if (e.isIntersecting) {
          e.target.classList.add("visible");
          observer.unobserve(e.target); // solo una vez
        }
      }),
      { threshold: 0.12 }
    );
    document.querySelectorAll(".reveal, .reveal-fade, .reveal-scale")
      .forEach((el) => observer.observe(el));
    return () => observer.disconnect();
  }, []);
}
```

Páginas donde aplicar:

Landing — cada sección de features con `delay-1/2/3` escalonado

Pricing — las dos tarjetas de plan entran desde abajo con 150ms de diferencia

Help / FAQ — cada tarjeta entra al hacer scroll

Changelog — cada versión entra al llegar al viewport

2. Tarjetas de viaje en el dashboard — microinteracciones

Ahora mismo `TripCardItem` no tiene apenas hover. Con solo CSS se puede conseguir mucho:

```
/* En globals.css o directo en el componente */
.trip-card {
  transition: transform 0.2s ease, box-shadow 0.2s ease, border-color 0.2s ease;
}
.trip-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 24px rgba(248, 113, 113, 0.12), 0 2px 6px rgba(15, 23, 42, 0.08);
  border-color: rgba(248, 113, 113, 0.3);
}
```

En el componente: añadir un indicador visual de días restantes o estado del viaje más prominente — actualmente es texto plano, podría ser una barra de progreso de la fecha del viaje (% del tiempo transcurrido).

3. Landing — hero con elemento flotante animado

El mockup de la app en el lado derecho del hero es una imagen estática. Animarlo crea percepción de producto vivo:

```
@keyframes float {
  0%, 100% { transform: translateY(0px) rotate(-1deg); }
  50%      { transform: translateY(-8px) rotate(-1deg); }
}

@keyframes float-delayed {
  0%, 100% { transform: translateY(0px) rotate(1deg); }
  50%      { transform: translateY(-6px) rotate(1deg); }
}

.hero-float { animation: float 4s ease-in-out infinite; }
.hero-float-2 { animation: float-delayed 5s ease-in-out infinite; }
```

Aplicar `hero-float` al contenedor del mockup. Sutilísimo (8px de desplazamiento) pero hace que la landing parezca viva.

4. Transición entre páginas — parallax al hacer scroll en la landing

Esta es la mejora que preguntas sobre "que salga la página como transición". La técnica más elegante y sin librerías externas es el **parallax en el hero**:

```
// En PublicLanding.tsx – efecto parallax en el hero
"use client";
import { useEffect, useRef } from "react";

// En el hero background (el blob coral)
const blobRef = useRef(null);

useEffect(() => {
  const onScroll = () => {
    if (!blobRef.current) return;
    const y = window.scrollY;
    blobRef.current.style.transform = `translateY(${y * 0.4}px)`;
  };
  window.addEventListener("scroll", onScroll, { passive: true });
  return () => window.removeEventListener("scroll", onScroll);
}, []);
```

El blob decorativo del hero se mueve más lento que el scroll → el texto parece "flotar" sobre él → efecto de profundidad sin librerías.

Para transición de sección a sección, añadir un `clip-path` animado en el borde inferior de cada sección:

```
.section-wave {
  clip-path: ellipse(100% 100% at 50% 0%);
}
/* La siguiente sección entra "por debajo" de la ola */
```

5. Pricing — animación de aparición del plan Popular

La tarjeta Premium ("Popular") debería destacar más visualmente:

```
@keyframes pulse-border {
  0%, 100% { box-shadow: 0 0 0 0 rgba(248,113,113,0.3); }
  50%      { box-shadow: 0 0 0 6px rgba(248,113,113,0); }
}

.plan-featured {
  animation: pulse-border 3s ease-in-out infinite;
}
```

Además, al llegar la tarjeta al viewport con scroll-reveal, hacer que entre con `scale(0.95)` → `scale(1)` da la sensación de que "aparece" con peso.

6. Actividades del plan — entrada con stagger

Cuando se carga la pestaña Plan, las actividades aparecen todas de golpe. Con un stagger de 50ms por tarjeta la sensación es mucho más premium:

```
// En TripPlanView – al montar cada día
items.map((item, idx) => (
  key={item.id}
  style={{ animationDelay: `${idx * 50}ms` }}
  className="reveal"
>
```

```
))
```

7. Header de la landing — transparente con blur al hacer scroll

Ahora el header tiene fondo sólido desde el primer momento. El efecto estándar en webs premium es: transparente en la parte superior → blur + fondo al hacer scroll.

```
// En PublicMarketingHeader.tsx
const [scrolled, setScrolled] = useState(false);

useEffect(() => {
  const onScroll = () => setScrolled(window.scrollY > 40);
  window.addEventListener("scroll", onScroll, { passive: true });
  return () => window.removeEventListener("scroll", onScroll);
}, []);

// En el className del header:
`sticky top-0 z-50 transition-all duration-300 ${
  scrolled
    ? "border-b border-slate-200/80 bg-white/90 backdrop-blur-md dark:border-[#1E293B] dark:bg-[#080C14]/90"
    : "bg-transparent border-transparent"
}`
```

8. Botones — efecto ripple o press al hacer click

Los botones primarios (coral) deberían dar feedback visual inmediato al pulsar:

```
.btn-primary {
  position: relative;
  overflow: hidden;
}

.btn-primary::after {
  content: '';
  position: absolute;
  inset: 0;
  background: rgba(255,255,255,0.2);
  opacity: 0;
  transition: opacity 0.15s;
}

.btn-primary:active::after {
  opacity: 1;
}
```

Además, añadir `active:scale-[0.97]` en Tailwind al botón primario da sensación táctil en móvil.

9. Formulario de crear viaje — transición entre pasos

El wizard de creación de viaje pasa de un paso al siguiente sin transición. Añadir un slide lateral:

```
@keyframes slide-in-right {
  from { opacity: 0; transform: translateX(20px); }
  to   { opacity: 1; transform: translateX(0); }
}

.step-enter { animation: slide-in-right 0.25s ease-out; }
```

10. Empty states — ilustraciones con animación

El empty state del dashboard (sin viajes) tiene un emoji estático. Con una animación sutil:

```
@keyframes bounce-soft {
  0%, 100% { transform: translateY(0); }
  50%     { transform: translateY(-6px); }
}

.empty-icon { animation: bounce-soft 2.5s ease-in-out infinite; }
```

Páginas específicas con mayor oportunidad de mejora

Landing (^/)

Sección	Mejora
Hero	Parallax en blob coral + mockup flotante
Stats (4 números)	Contador animado: 0 → número final al entrar al viewport
Features (grid 3 cols)	Scroll reveal escalonado con delay-1/2/3
Testimoniales	Fade in desde los lados (izquierda/derecha alternado)
CTA final	Scale-in al llegar al viewport

Dashboard (^/dashboard^)

Elemento	Mejora
Tarjetas de viaje	Hover elevado + borde coral suave
Sección vacía	Icono animado con bounce
Sección "Próximos viajes"	Shimmer skeleton mientras carga

Resumen del viaje (^/trip/[id]/summary^)

Elemento	Mejora
Hero oscuro	Ya tiene blur circles — añadir animación muy sutil de rotación lenta
Módulos de navegación	Hover con scale(1.02) en vez de solo color

Pricing (/pricing)

Elemento	Mejora
Tarjeta Popular	Pulse en el borde + scroll-reveal con scale
Tabla comparativa	Fade-in por filas con stagger

Librerías a considerar (todas pequeñas)

Librería	Tamaño	Para qué
Nativa (CSS + JS)	0kb	Scroll reveal, parallax, microinteracciones
`motion` (Framer Motion lite)	~15kb	Transiciones de página más complejas
`@number-flow/react`	~4kb	Contadores animados (para stats de la landing)
`canvas-confetti`	~15kb	Celebración al completar el primer viaje

Recomendación: empezar con todo nativo (CSS + IntersectionObserver) — cubre el 80% de las mejoras sin añadir dependencias. Solo si se quieren transiciones de página complejas, considerar `motion`.

Orden de implementación recomendado

- Semana 1 – Sin dependencias, máximo impacto:
- `globals.css`: añadir `@keyframes` (`slide-up`, `fade-in`, `float`)
 - `hooks/useScrollReveal.ts`: `IntersectionObserver`
 - Landing: scroll reveal en features + testimoniales
 - Header marketing: transparente → blur al scroll
 - Tarjetas de viaje: hover elevado con sombra coral
- Semana 2 – Interacciones internas:
- Plan: stagger de 50ms en actividades al cargar
 - Pricing: pulse en tarjeta Popular + scroll reveal
 - Hero landing: parallax en blob + mockup flotante
 - Botones: `active:scale-[0.97]` + ripple en `:active`
- Semana 3 – Polish:
- Contadores animados en los stats de la landing
 - Transición entre pasos del wizard de crear viaje
 - Empty states con animación
 - Confetti al crear el primer viaje (`canvas-confetti`)

Lo que NO hacer

No añadir Framer Motion todavía — las animaciones CSS son suficientes y 0kb de bundle

No animar demasiado — máximo 2-3 elementos animados por pantalla simultáneamente

No usar `transition: all` — solo animar las propiedades necesarias (transform, opacity, box-shadow)

No animar en `prefers-reduced-motion` — siempre añadir:

```
`css
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

Análisis de diseño generado en Junio 2026 sobre Kaviro-main